

◆ Desugaring Kotlin features and their performance impact

by Ionuț Baloșin

ionutbalosin.com

 [@ionutbalosin](https://twitter.com/ionutbalosin)

Copyright © 2019 by IonuțBaloșin

About Me

Software Architect



Technical Trainer

SOFTWARE ARCHITECTURE

JAVA PERFORMANCE AND TUNING

Writer

IonutBalosin.com



Speaker



Today's Content

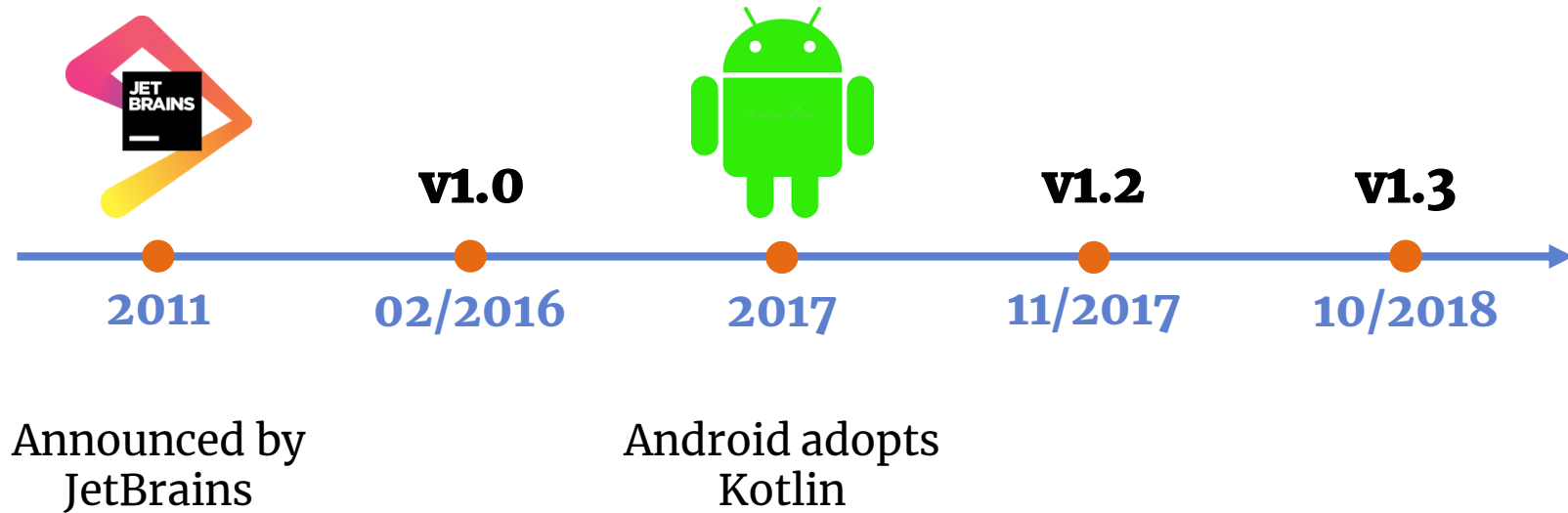
- 01 **Intro**
- 02 **Null Safety**
- 03 **String Interpolation**
- 04 **[data] Classes**
- 05 **Extension Functions**
- 06 **Default Arguments**
- 07 **Inlining**

Intro

01



“Cross-platform, statically typed, general-purpose programming language, fully interoperable with Java.”



Why Kotlin?



“To create a better language than Java.”

Andrey Breslav – Lead Language Designer



CONCISE



SAFE



INTEROPERABLE

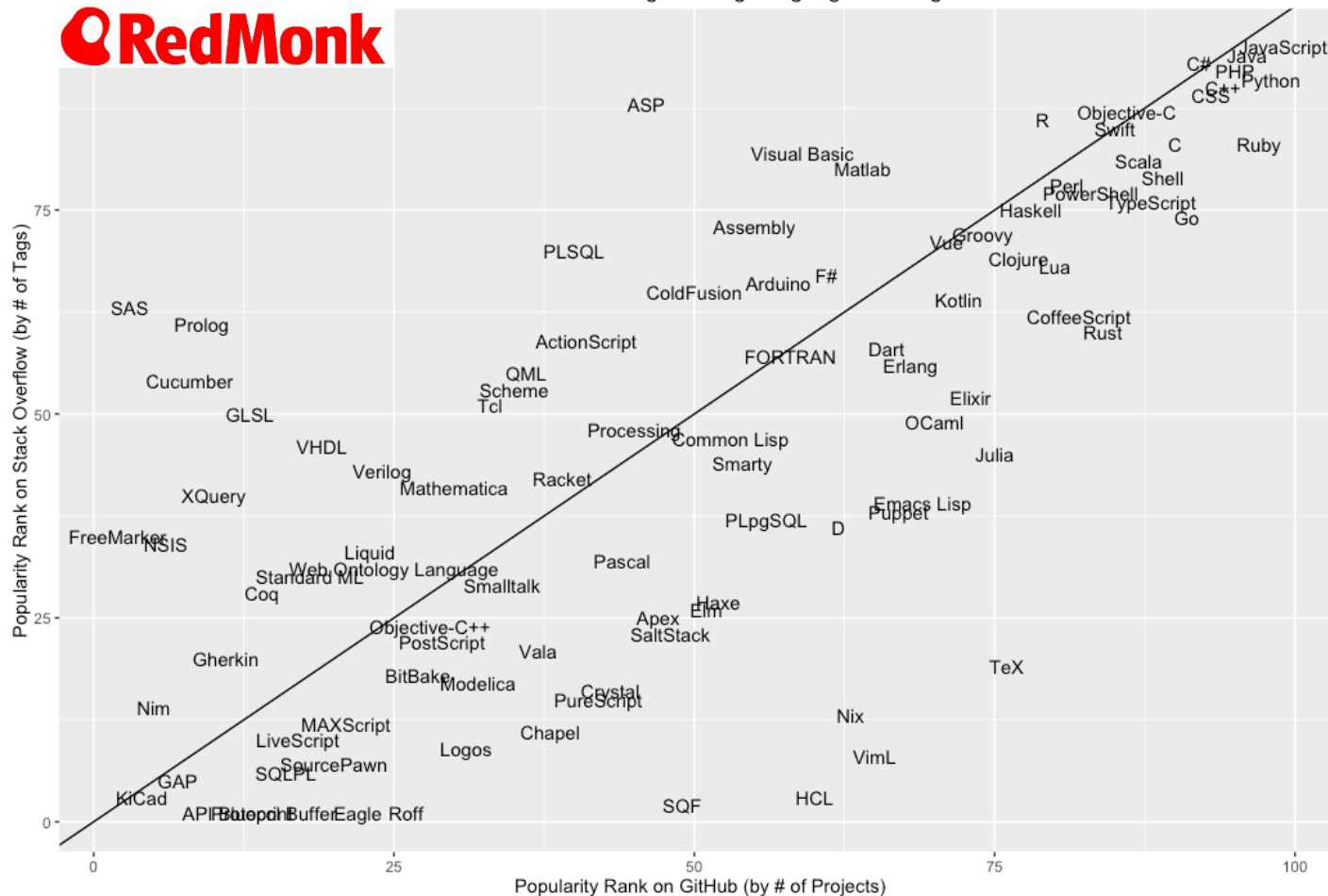


RICH FEATURES

March 2019	Language	Ratings
1	Java	14.880%
2	C	13.305%
3	Python	8.262%
...		
16	Groovy	1.178%
...		
28	Scala	0.467%
...		
39	Kotlin	0.288%
...		
62	Clojure	n/a

March 2019	Language	Ratings
1	Java	14.880%
2	C	13.305%
3	Python	8.262%
...		
16	Groovy	1.178%
...		
28	Scala	0.467%
...		
39	Kotlin	0.288%
...		
62	Clojure	n/a

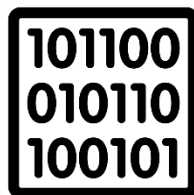
RedMonk Q318 Programming Language Rankings





.kt source

Kotlin compiler



bytecode

Null Safety

02

(1) NOT NULLABLE

```
var <notNullVar>: Type = Value
```

```
lateinit var <notNullVar>: Type = Value
```

```
READ <notNullVar>
```

```
var notNullString: String = "conference"
```

```
return notNullString.length
```

```
var notNullString: String = "conference"
```

```
return notNullString.length
```

```
String notNullString = "conference";
```

```
return notNullString.length();
```

```
lateinit var lateInitString: String
```

```
// ... optional initializer ...
```

```
return lateInitString.length
```



```
lateinit var lateInitString: String
```

```
// ... optional initializer ...
```

```
return lateInitString.length
```

```
if (lateInitString == null) {  
    Intrinsic.throwUninitializedPropertyAccessException("...");  
}  
  
return lateInitString.length();
```

(2) NULLABLE

 nullable reference

```
var <maybeNullVar>: Type? = Value
```

```
READ <maybeNullVar>?      // NULL safety
```

```
READ <maybeNullVar>!!     // throws NULL
```

```
var maybeNullString: String? = initString() // possible NULL
```

```
return maybeNullString?.length
```

↑
— NULL safety operator

```
var maybeNullString: String? = initString() // possible NULL
```

```
return maybeNullString?.length
```

↑ NULL safety operator

```
String var10000 = maybeNullString;
```

```
return var10000 != null ? var10000.length() : null;
```

```
var maybeNullString: String? = initString() // possible NULL
```

```
return maybeNullString!!.length
```

↑
Double-bang operator

```
var maybeNullString: String? = initString() // possible NULL
```

```
return maybeNullString!!.length
```

↑ Double-bang operator

```
if (maybeNullString == null) {  
    Intrinsics.throwNpe();  
}
```

```
return maybeNullString.length();
```

```
@Benchmark
fun notNull(): Int {
    return notNullString.length;    // "conference".length
}
```

```
@Benchmark
fun lateInit(): Int {
    return lateInitString.length;    // "conference".length
}
```

```
@Benchmark
fun maybeNull(): Int? {
    return maybeNullString?.length;    // "conference".length
}
```


	<i>Average Time (ns/op)</i>
notNull	2.895 ± 0.202
lateInit	2.844 ± 0.163
maybeNull	3.793 ± 0.577

Java Runtime Environment 11.0.2+9-LTS

	Average Time (ns/op)
notNull	2.895 ± 0.202
lateInit	2.844 ± 0.163
maybeNull	3.793 ± 0.577



Java Runtime Environment 11.0.2+9-LTS

```
fun notNull(): Int { ... }
```

```
fun lateInit(): Int { ... }
```

```
fun maybeNull(): Int? { ... }
```

```
public final int notNull() { ... }
```

```
public final int lateInit() { ... }
```

```
public final Integer maybeNull() { ... }
```

```
public final Integer maybeNull() { // Kotlin decompiled
    String var10000 = this.maybeNullString;
    return var10000 != null ? var10000.length() : null;
}
```



```
public final Integer maybeNull() { // Kotlin decompiled  
    String var10000 = this.maybeNullString;  
    return var10000 != null ? var10000.length() : null;  
}
```

Integer int



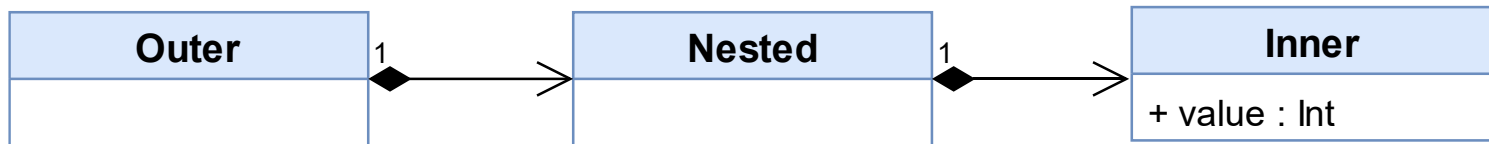
 **NULLABLE REFERENCES** for “*primitive wrappers*” (e.g. Int?) prevents Kotlin compiler to desugar them into “*Java primitive*” value types (e.g. int).

As a side effect, it increases the performance penalty due additional boxing effect!

 **NOT NULLABLE REFERENCES** for “*primitive wrappers*” are desugared by Kotlin compiler into “*Java primitives*”!

Elvis Operator





@Benchmark

```
fun elvis() : Int? {  
    return outer?.nested?.inner?.value ?: aCustomValue;  
}
```

↑ Elvis operator


```
public final Integer elvis() { // Kotlin decompiled
    if (outer != null) {
        Nested var1 = outer.getNested();
        if (var1 != null) {
            Inner var2 = var1.getInner();
            if (var2 != null) {
                Integer var3 = var2.getValue();
                if (var3 != null) {
                    return var3;
                }
            }
        }
    }
    return aCustomValue;
}
```



	Average Time (ns/op)
Elvis operator	4.154 ± 0.205
Optional	4.201 ± 0.235

Java Runtime Environment 11.0.2+9-LTS

No significant difference in performance.

Optional is slightly slower due to extra INVOKESTATIC calls (e.g. ofNullable()).

```
public final Integer elvis() // assembly based on Kotlin source
```

```
mov    ecx,DWORD PTR [rbp+0x14]          ;*getfield outer  
mov    r10d,DWORD PTR [r12+rcx*8+0xc]    ;*getfield nested  
                                              ; implicit exception  
mov    r11d,DWORD PTR [r12+r10*8+0xc]    ;*getfield inner  
                                              ; implicit exception  
mov    ecx,DWORD PTR [r12+r11*8+0xc]    ;*getfield value  
                                              ; implicit exception  
mov    ebp,DWORD PTR [r12+rcx*8+0xc]    ;*getfield value  
                                              ; - java.lang.Integer::intValue@1  
                                              ; implicit exception
```

String Interpolation

03

```
"$shape area is ${length * width} $unit"
```

```
"$quantity kilogram of $fruits costs $price $currency"
```

String interpolation evaluates a literal and replaces the placeholders by values.
String interpolation $\approx$ variable interpolation $\approx$ variable substitution

```
"$shape area is ${length * width} $unit"
```

```
"$quantity kilogram of $fruits costs $price $currency"
```

```
shape + " area is " + length * width + ' ' + unit
```

```
quantity + " kilogram of " + fruits + " costs " + price + ' ' +  
currency
```

"\$shape area is \${length * width} \$unit" // Java 8 bytecode

```
GETFIELD shape : LString;
INVOKEVIRTUAL StringBuilder.append (LString;) LStringBuilder;

LDC " area is "
INVOKEVIRTUAL StringBuilder.append (LString;) LStringBuilder;

GETFIELD length : D
GETFIELD width : D
DMUL
INVOKEVIRTUAL StringBuilder.append (D) LStringBuilder;

BIPUSH 32 ; space
INVOKEVIRTUAL StringBuilder.append (C) LStringBuilder;

GETFIELD unit : LString;
INVOKEVIRTUAL StringBuilder.append (LString;) LStringBuilder;
```

```
"$shape area is ${length * width} $unit" // Java 8 bytecode
```

```
GETFIELD shape : LString;
```

```
INVOKEVIRTUAL StringBuilder.append (LString;)LStringBuilder;
```

```
LDC " area is "
```

```
INVOKEVIRTUAL StringBuilder.append (LString;)LStringBuilder;
```

**Kotlin compiler relies on `StringBuilder` to
handle `String` interpolation!**

```
GETFIELD width : D
```

```
DMUL
```

```
INVOKEVIRTUAL StringBuilder.append (D)LStringBuilder;
```

```
BIPUSH 32 ; space
```

```
INVOKEVIRTUAL StringBuilder.append (C)LStringBuilder;
```

```
GETFIELD unit : LString;
```

```
INVOKEVIRTUAL StringBuilder.append (LString;)LStringBuilder;
```


@Benchmark

```
fun shapeArea(): String {  
    return "${shape} area is ${length * width} $unit"  
}
```

@Benchmark

```
fun fruitsPrice(): String {  
    return "$quantity kilogram of $fruits costs $price $currency"  
}
```



Average Time (ns/op)	
shapeArea	fruitsPrice
76.330 ± 4.574	126.436 ± 6.504
62.774 ± 4.691	91.004 ± 4.828

Java Runtime Environment 11.0.2+9-LTS

Java benchmark uses “+” operand for String concatenation, similar to what Kotlin decompiled code looks like.



Average Time (ns/op)	
shapeArea	fruitsPrice
76.330 ± 4.574	126.436 ± 6.504
62.774 ± 4.691	91.004 ± 4.828



Java Runtime Environment 11.0.2+9-LTS

Java benchmark uses “+” operand for String concatenation, similar to what Kotlin decompiled code looks like.

```
"$shape area is ${length * width} $unit" // Java 11 bytecode
```

```
GETFIELD shape : LString;
```

```
GETFIELD length : D
```

```
GETFIELD width : D
```

```
DMUL
```

```
GETFIELD unit : LString;
```

```
INVOKEDYNAMIC makeConcatWithConstants (LString;DLString;)LString; [
```

```
    // handle kind 0x6 : INVOKESTATIC
```

```
    invoke/StringConcatFactory.makeConcatWithConstants(
```

```
    Linvoke/MethodHandles$Lookup;LString;Linvoke/MethodType;LString;[LObject;
```

```
    )Linvoke/CallSite;
```

```
    // arguments:
```

```
    "\u0001 area is \u0001 \u0001"
```

```
]
```

```
"$shape area is ${length * width} $unit" // Java 11 decompiled
```

```
GETFIELD shape : LString;
```

```
GETFIELD length : D
```

```
GETFIELD width : D
```

```
DI
```

```
GETFIELD unit : LString;
```

```
INVOKEDYNAMIC makeConcatWithConstants(LString;LString;)LString;
```

```
// handle kind 0x6 : INVOKEDYNAMIC
```

```
invoke/stringconcatfactory.makeConcatWithConstants(
```

```
Linvoke/MethodHandles$Lookup;LString;Linvoke/MethodType;LString;[LObject;
```

```
)Linvoke/CallSite;
```

```
// arguments:
```

```
"\u0001 area is \u0001 \u0001"
```

```
]
```

Starting JDK 9 (i.e. JEP 280: Indify String Concatenation), Java replaces String concatenation bytecode sequence (i.e. StringBuilder) by INVOKEDYNAMIC

❗ **Kotlin compiler** generates **Java 8** compliant bytecode, hence it cannot take advantage of latest bytecode improvements added starting **Java 9**.

E.g.

JEP 280: Indify String Concatenation (JDK9)

JEP 309: Dynamic Class-File Constants (JDK11)

JEP 181: Nest-Based Access Control (JDK11)

[data] Classes

04

```
class Conference(  
    val name: String,           // not nullable  
    val city: String?,         // nullable  
    val year: Int               // not nullable  
)
```



```
class Conference(  
    val name: String,           // not nullable  
    val city: String?,         // nullable  
    val year: Int               // not nullable  
)
```

```
final class Conference = {  
    fields, getters, constructor  
}
```

```

public final class Conference { // Kotlin decompiled
    @NotNull
    private final String name;
    @Nullable
    private final String city;
    private final int year;      <- converted to primitive
    @NotNull
    public final String getName() { /*...*/ }
    @Nullable
    public final String getCity() { /*...*/ }
    public final int getYear() { /*...*/ }
    public Conference(@NotNull String name, String city, int year) {
        Intrinsic.checkParameterIsNotNull(name, "name");
        super();
        // ...
    }
}

```

```

public final class Conference { // Kotlin decompiled
    @NotNull
    private final String name;
    @Nullable
    private final String city;
    private final int year; <- converted to primitive

    @NotNull
    public final String getName() { /*...*/ }

    @Nullable
    public final String getCity() { /*...*/ }

    public final int getYear() { /*...*/ }

    public Conference(@NotNull String name, String city, int year) {
        Intrinsics.checkParameterIsNotNull(name, "name");
        super();
        // ...
    }
}

```

```

public final class Conference { // Kotlin decompiled
    @NotNull
    private final String name;
    @Nullable
    private final String city;
    private final int year; <- converted to primitive
    @NotNull
    public final String getName() { /*...*/ }
    @Nullable
    public final String getCity() { /*...*/ }
    public final int getYear() { /*...*/ }

    public Conference(@NotNull String name, String city, int year) {
        Intrinsics.checkParameterIsNotNull(name, "name");
        super();
        // ... setters ...
    }
}

```

```
data class Talk(  
    val name: String,        // not nullable  
    val rating: Double?      // nullable  
)
```

```
data class Talk(  
    val name: String,        // not nullable  
    val rating: Double?      // nullable  
)
```

```
final class Talk = {  
    fields, getters, constructor,  
    component functions, copy, synthetic copy,  
    toString, equals, hashCode  
}
```

```

public final class Talk { // Kotlin decompiled
    @NotNull
    public final String component1() { return this.name; }

    @Nullable
    public final Double component2() { return this.rating; }

    @NotNull
    public final Talk copy(@NotNull String name,
                           @Nullable Double rating) {
        Intrinsic.checkParameterIsNotNull(name, "name");
        return new Talk(name, rating);
    }

    @NotNull // $FF: synthetic method
    public static Talk copy$default(Talk var0,
                                    String name, Double rating,
                                    int var3, Object var4) {
        if ((var3 & 1) != 0) { name = var0.name; }
        if ((var3 & 2) != 0) { rating = var0.rating; }
        return var0.copy(name, rating); }
}

```

```
public final class Talk { // Kotlin decompiled
```

```
    @NotNull
```

```
    public final String component1() { return this.name; }
```

```
    @Nullable
```

```
    public final Double component2() { return this.rating; }
```

```
@NotNull
```

```
public final Talk copy(@NotNull String name,  
                       @Nullable Double rating) {  
    Intrinsics.checkNotNull(name, "name");  
    return new Talk(name, rating);  
}
```

```
@NotNull // $FF: synthetic method
```

```
public static Talk copy$default(Talk var0,  
                                String name, Double rating,  
                                int var3, Object var4) {  
    if ((var3 & 1) != 0) { name = var0.name; }  
    if ((var3 & 2) != 0) { rating = var0.rating; }  
    return var0.copy(name, rating); }
```

```
}
```



```
public final class Talk { // Kotlin decompiled
```

```
@NotNull
```

```
public final String component1() { return this.name; }
```

```
@Nullable
```

```
public final Double component2() { return this.rating; }
```

```
@NotNull
```

```
public final Talk copy(@NotNull String name,  
                        @Nullable Double rating) {  
    Intrinsics.checkParameterIsNotNull(name, "name");  
    return new Talk(name, rating);  
}
```

```
@NotNull // $FF: synthetic method
```

```
public static Talk copy$default(Talk var0,  
                                String name, Double rating,  
                                int var3, Object var4) {  
    if ((var3 & 1) != 0) { name = var0.name; }  
    if ((var3 & 2) != 0) { rating = var0.rating; }  
    return var0.copy(name, rating); }
```

↑ calls the user defined function

```
val talk = Talk("Kotlin under the hood", 10.0)  
  
val talkCopyV1 = talk.copy(name = "C++", rating = 9.5)
```

```
val talk = Talk("Kotlin under the hood", 10.0)

val talkCopyV1 = talk.copy(name = "C++", rating = 9.5)
```

```
Talk talkCopyV1 = talk.copy("C++", 9.5D);
```

```
val talk = Talk("Kotlin under the hood", 10.0)

val talkCopyV1 = talk.copy(name = "C++", rating = 9.5)

val talkCopyV2 = talk.copy()
val talkCopyV3 = talk.copy(name = "C++")
val talkCopyV4 = talk.copy(rating = 9.5)
```

```
Talk talkCopyV1 = talk.copy("C++", 9.5D);

Talk talkCopyV2 = Talk.copy$default(talk, null, null, 3, null);
Talk talkCopyV3 = Talk.copy$default(talk, "C++", null, 2, null);
Talk talkCopyV4 = Talk.copy$default(talk, null, 9.5D, 1, null);
```



var3

```
val talk = Talk("Kotlin under the hood", 10.0)

// Destructuring declarations
val (name, rating) = talk           // or a function return
```

```
val talk = Talk("Kotlin under the hood", 10.0)
```

```
// Destructuring declarations
```

```
val (name, rating) = talk           // or a function return
```

```
String name = talk.component1();
```

```
Double rating = talk.component2();
```

Extension Functions

05

```
<accessModifier> fun ReceiverType.<funcName>(): ReturnType {  
    // ... body ...  
}
```

Extension functions extend a class with new functionality without having to inherit from the class or use any type of design pattern such as Decorator.


```
// ExtensionFunction.kt file  
public fun String.wrap(): String = "< $this >"
```

```
// ExtensionFunction.kt file
public fun String.wrap(): String = "< $this >"

"Welcome to Java Conference".wrap()
```

```
// ExtensionFunction.kt file
public fun String.wrap(): String = "< $this >"

"Welcome to Java Conference".wrap()
```

```
public final class ExtensionFunctionKt {
    public static final String wrap(@NotNull String $receiver) {
        return "< " + $receiver + " >";
    }

    wrap("Welcome to Java Conference");
}
```

Default Arguments

06

```
<accessModifier> fun <funcName> (  
    <param1>: Type,  
    <param2>: Type = defaultValue,  
    <param3>: Type = defaultValue) : ReturnType {  
    // ... body ...  
}
```

Function parameters can have default values, which are used when a corresponding argument is omitted.

```
private fun expression(  
    value: Int,  
    factor: Int = 1,  
    delta: Int = 0): Int  
= value * factor + delta
```

```
private fun expression(  
    value: Int,  
    factor: Int = 1,  
    delta: Int = 0): Int  
= value * factor + delta
```

```
private final int expression( ... ) { ... }
```

```
// $FF: synthetic method
```

```
static int expression$default( ... ) { ... }
```

```

// $FF: synthetic method added by Kotlin compiler
static int expression$default(
    DefaultArguments var0,           // enclosing instance
    int value, int factor, int delta, // user defined parameters
    int var4, Object var5) {
    if (var5 != null) {
        throw new UnsupportedOperationException("...");
    } else {
        if ((var4 & 2) != 0) { factor = 1; }
        if ((var4 & 4) != 0) { delta = 0; }
        return var0.expression(value, factor, delta);
    }
}

```

↑ calls the user defined function


```
expression(8) // factor = 1, delta = 0
```

```
expression(8, factor = 2) // delta = 0
```

```
expression(8, factor = 2, delta = 1)
```

```
expression$default(this, 8, 0, 0, 6, null);
```

```
expression$default(this, 8, 2, 0, 4, null);
```

```
expression(8, 2, 1);
```



var4

@Benchmark

```
fun default2Arguments(): Int {  
    return expression(value)  
}
```

@Benchmark

```
fun default1Argument(): Int {  
    return expression(value, factor = factor)  
}
```

@Benchmark

```
fun noDefault(): Int {  
    return expression(value, factor = factor, delta = delta)  
}
```

	Average Time (ns/op)
default2Arguments	2.445 ± 0.096
default1Argument	2.456 ± 0.145
noDefault	2.869 ± 0.156

Java Runtime Environment 11.0.2+9-LTS

```
fun default2Arguments() // value * 1 + 0
    mov     eax,DWORD PTR [rsi+0xc]    ;*getfield value

fun default1Argument() // value * factor + 0
    mov     eax,DWORD PTR [rsi+0x10]
    imul    eax,DWORD PTR [rsi+0xc]    ;*imul

fun noDefault() // value * factor + delta
    mov     eax,DWORD PTR [rsi+0x10]
    imul    eax,DWORD PTR [rsi+0xc]    ;*imul
    add     eax,DWORD PTR [rsi+0x14]    ;*iadd
```

JIT Compiler devirtualize and inlines the calls, removing the synthetic method overhead.

Inlining

07

```
<accessModifier> fun <highOrderFuncName> (  
    <param>: Type,  
    <func>: (Type) -> ReturnType  
): ReturnType {  
    // ... body ...  
}
```



function type

A **higher-order function** takes functions as parameters or returns a function.
A **function type** represents the type of a that function. E.g. $(R, T) \rightarrow R$.

```
private fun sumOfSquares (
    max: Int,
    square: (Int) -> Long // function
): Long {
    var sum = 0L
    for (i in 1..max) {
        sum += square(i)
    }
    return sum
}
```

```
private fun sumOfSquares(  
    max: Int,  
    square: (Int) -> Long // function  
) : Long {  
    var sum = 0L  
    for (i in 1..max) {  
        sum += square(i)  
    }  
    return sum  
}
```

```
private final long sumOfSquares(int max, Function1 func) { ... }
```



```
/** A function that takes 1 argument. */  
public interface Function1<in P1, out R> : Function<R> {  
    /** Invokes the function with the specified argument. */  
    public operator fun invoke(p1: P1): R  
}
```



calls square(Int)

Function types are implemented as interfaces under the hood.
Instantiating a function type creates an instance of the Function interface.
Kotlin standard library defines generic function type of up to 22 (e.g. **Function22**)

// Kotlin decompiled

```
private final long sumOfSquares(int max, Function1 func) {  
    long sum = 0L; int i = 1; int var7 = max;  
    if (i <= max) {  
        while(true) {  
            sum += ((Number) func.invoke(i)).longValue();  
            if (i == var7) {  
                break;  
            }  
            ++i;  
        }  
    }  
    return sum;  
}
```

interface
↓

↑
calls square(Int)

```
@Param("1_000_000")  
  
private var param: Int = 0  
  
@Benchmark  
  
fun sumOfSquares(): Long {  
    return sumOfSquares(param) {  
        it * it;  
    }  
}
```

sumOfSquares	
average (ms/op)	4.668 ± 0.093
gc.alloc.rate (MB/sec)	4,358.972 ± 86.090
gc.count	229
gc.time (ms)	190

Java Runtime Environment 11.0.2+9-LTS

sumOfSquares	
average (ms/op)	4.668 ± 0.093
gc.alloc.rate (MB/sec)	4,358.972 ± 86.090
gc.count	229
gc.time (ms)	190



Java Runtime Environment 11.0.2+9-LTS

How could the performance be improved?

Inlining



```
inline private fun sumOfSquares(  
    max: Int,  
    square: (Int) -> Long // function  
) : Long {  
    // ... body ...  
}
```

“**inline**” makes the function itself and the lambda passed to it to be inlined at the call site.

	sumOfSquares	sumOfSquares (inlined)
average (ms/op)	4.668 ± 0.093	0.379 ± 0.010
gc.alloc.rate (MB/sec)	4358.9	$\approx 10^{-4}$
gc.count	229.0	≈ 0
gc.time (ms)	190.08	≈ 0

Java Runtime Environment 11.0.2+9-LTS

	sumOfSquares	sumOfSquares (inlined)
average (ms/op)	4.668 ± 0.093	0.379 ± 0.010
gc.alloc.rate (MB/sec)	4358.9	$\approx 10^{-4}$
gc.count	229.0	≈ 0
gc.time (ms)	190.08	≈ 0

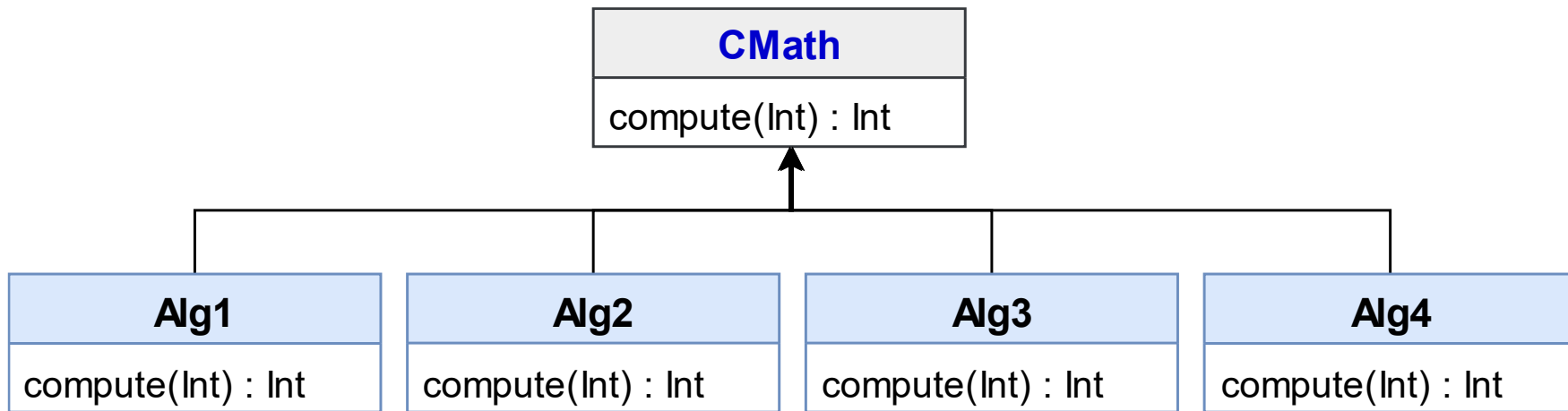


Java Runtime Environment 11.0.2+9-LTS

Inlining high-level function at call-sites saves the lambda objects to be created, which in essence means less memory allocations and virtual calls.

Polymorphism





```
inline fun compute(math: CMath, i: Int): Int {  
    return math.compute(i) // return param * constant  
}
```

@Benchmark

```
fun monomorphicCall(): Int { // param * 17
    return compute(alg1, param)
}
```

@Benchmark

```
fun bimorphicCall(): Int { // param * 17 + param * 19
    return compute(alg1, param) + compute(alg2, param)
}
```

@Benchmark

```
fun megamorphic3Call(): Int { // param * 17 + param * 19 + param * 23
    return compute(alg1, param) + compute(alg2, param) + compute(alg3, param)
}
```

@Benchmark

```
fun megamorphic4Call(): Int { // param * 17 + param * 19 + param * 23 + param * 29
    return compute(alg1, param) + compute(alg2, param) + compute(alg3, param) +
        compute(alg4, param)
}
```

	Average Time (ns/op)	
		
monomorphicCall	3.439 ± 0.198	2.905 ± 0.048
bimorphicCall	3.776 ± 0.213	4.092 ± 0.243
megamorphic3Call	4.081 ± 0.158	10.142 ± 0.509
megamorphic4Call	4.441 ± 0.295	11.545 ± 0.675

Java Runtime Environment 11.0.2+9-LTS

	Average Time (ns/op)	
		
monomorphicCall	3.439 ± 0.198	2.905 ± 0.048
bimorphicCall	3.776 ± 0.213	4.092 ± 0.243
megamorphic3Call	4.081 ± 0.158	10.142 ± 0.509
megamorphic4Call	4.441 ± 0.295	11.545 ± 0.675



Java Runtime Environment 11.0.2+9-LTS

public int megamorphic3Call() // assembly based on Java source

mov	edx,DWORD PTR [r10+0x70]	;*getstatic param	}	param * 17
shl	rsi,0x3	;*getfield alg1		
call	0x00007f6a981b8400	;*invokestatic compute		
mov	edx,DWORD PTR [r11+0x70]	;*getstatic param	}	param * 19
call	0x00007f6a981b8400	;*invokestatic compute		
mov	ebp,eax		}	+
add	ebp,DWORD PTR [rsp+0x58]	;*iadd		
mov	edx,DWORD PTR [r10+0x70]	;*getstatic param	}	param * 23
shl	rsi,0x3	;*getfield alg3		
call	0x00007f6a981b8400	;*invokestatic compute		
mov	edx,eax		}	+
add	edx,ebp	;*iadd		

fun megamorphic3Call() *// assembly based on Kotlin source*

```
mov    r8d,DWORD PTR [r10+0xc]      ;*getfield alg1
mov    r11d,DWORD PTR [r10+0x74]    ;*getstatic param
mov    r10d,DWORD PTR [r10+0x10]    ;*getfield alg2

mov    r8d,r11d
shl    r8d,0x4
add    r8d,r11d                      ;*imul      } param * 17

mov    r10d,DWORD PTR [r10+0x14]    ;*getfield alg3
imul   r9d,r11d,0x13
add    r9d,r8d                      ;*iadd      } + param * 19

imul   edx,r11d,0x17
add    edx,r9d                      ;*iadd      } + param * 23
```

```
fun megamorphic3Call() // assembly code
```

```
mov     r8d,DWORD PTR [r10+0xc]      ;*getfield alg1
mov     r11d,DWORD PTR [r10+0x74]    ;*getstatic param
mov     r10d,DWORD PTR [r10+0x10]    ;*getfield alg2
```

```
mov     r8d,r11d
```

Kotlin explicit inlining pays off in performance,
especially at "megamorphic" call-sites.

```
add     r8d,r11d                    ;*iadd
mov     r10d,DWORD PTR [r10+0x14]    ;*getfield alg3
```

```
imul    r9d,r11d,0x13
```

```
add     r9d,r8d                     ;*iadd
```

+ param x 29

```
imul    edx,r11d,0x17
```

```
add     edx,r9d                     ;*iadd
```

+ param x 23

Conclusions

- 01 Intro
- 02 Null Safety ✓
- 03 String Interpolation !
- 04 [data] Classes ✓
- 05 Extension Functions ✓
- 06 Default Arguments ✓
- 07 Inlining ✓ !

The background of the image is a deep space scene featuring a dense field of stars and several prominent spiral galaxies. The galaxies are rendered in shades of blue, purple, and pink, with bright, glowing cores. The overall color palette is dark, with the light from the stars and galaxies providing the primary illumination.

◆ Thanks
Mulțumesc
Danke

Resources

Slides

<https://ionutbalosin.com/talks>

Source Code

<https://github.com/ionutbalosin/kotlin-vs-java-benchmarks>

ionutbalosin.com
 @ionutbalosin

Copyright © 2019 by IonuțBaloșin

A deep space background featuring a dense field of stars and several prominent spiral galaxies. The galaxies are illuminated with vibrant colors, including deep reds, purples, and blues, set against the dark void of space.

◆ Bonus Slides

Spread Operator


```
var array: Array<Int> // Integer[] {Elem1, Elem2, Elem3, ...}
```

```
List <- asList( *array /* , Y1, Y2, Y3, ... */ )
```

 **spread operator**

Spread operator (*) can only be used when passing arguments to a function.

```
var array: Array<Int> // Integer[] {Elem1, Elem2, Elem3, ...}
```

```
List <- asList( *array /* , Y1, Y2, Y3, ... */ )
```



[Elem1, Elem2, Elem3 ...]

Spread operator (*) can only be used when passing arguments to a function.

```
var array: Array<Int> // Integer[] {Elem1, Elem2, Elem3, ...}
```

```
List <- asList( *array /* , Y1, Y2, Y3, ... */ )
```


[Elem1, Elem2, Elem3 ...]

```
Arrays.asList(  
    (Integer[]) Arrays.copyOf(array, array.length)  
);
```

```
var array: IntArray // int[]  
  
// varArgFunction(vararg data: Int) { ... }  
  
CALL varArgFunction(*array)
```

Using **spread operator (*)** to pass arguments while calling a vararg function.

```
var array: IntArray // int[]
```

```
// varArgFunction(vararg data: Int) { ... }
```

```
CALL varArgFunction(*array)
```

```
varArgFunction(  
    Arrays.copyOf(array, array.length)  
);
```

```

@Param("0", "10", "100", "1000", "10000")
private var size: Int = 0 // array size

var arrayOfInteger: Array<Int>
var intArray: IntArray

@Benchmark
fun asList(): MutableList<Int> {
    return asList(*arrayOfInteger)
}

@Benchmark
fun varArgFunction(blackHole: Blackhole) {
    return varArgFunction(blackHole, *intArray)
}

private fun varArgFunction(blackHole: Blackhole, vararg data: Int) {
    blackHole.consume(data)
}

```

	Average Time (ns/op)	
array size	asList	varArgFunction
0	7.755 ± 0.324	5.444 ± 0.350
10	11.718 ± 0.653	8.930 ± 0.479
100	61.659 ± 3.905	55.658 ± 0.879
1000	552.970 ± 18.188	554.532 ± 12.782
10000	5,339.638 ± 183.985	5,418.889 ± 149.093

Java Runtime Environment 11.0.2+9-LTS

Performance is degrading with the array size.

	Average Time (ns/op)	
array size	asList	varArgFunction
0	7.755 ± 0.324	5.444 ± 0.350
10	11.718 ± 0.653	8.930 ± 0.479
100	61.659 ± 3.905	55.658 ± 0.879
1000	552.970 ± 18.188	554.532 ± 12.782
10000	5,339.638 ± 183.985	5,418.889 ± 149.093

Java Runtime Environment 11.0.2+9-LTS

Performance is degrading with the array size.

 **Avoid using Kotlin spread operator!**

Tail Recursion

```
private tailrec fun fibonacci(n: Int, a: Int, b: Int): Int =  
    if (n == 0)  
        a;  
    else if (n == 1)  
        b;  
    else fibonacci(n - 1, b, a + b);
```

A **tail recursive** function is one where every recursive call is the last thing done by the function before returning.

```

private final int fibonacci(int n, int a, int b) { // Kotlin decompiled
    while(true) {
        int var10000;
        if (n == 0) {
            var10000 = a;
        } else {
            if (n != 1) {
                int var10001 = b; var10000 = n - 1;
                b += a;  a = var10001;  n = var10000;
                continue;
            }
            var10000 = b;
        }
        return var10000;
    }
}

```

$$\left. \begin{array}{l} \text{aux} = F_n \\ F_n = F_n + F_{n-1} \\ F_{n-1} = \text{aux} \end{array} \right\}$$

	Average Time (us/op)
fibonacciRec	19.671 ± 0.386
fibonacciTailRec	3.673 ± 0.076

Java Runtime Environment 11.0.2+9-LTS

Computing the $N=10,000^{\text{th}}$ Fibonacci element.

For the recursive function, performance is degrading with the size of N .

	Average Time (us/op)
fibonacciRec	19.671 ± 0.386
fibonacciTailRec	3.673 ± 0.076

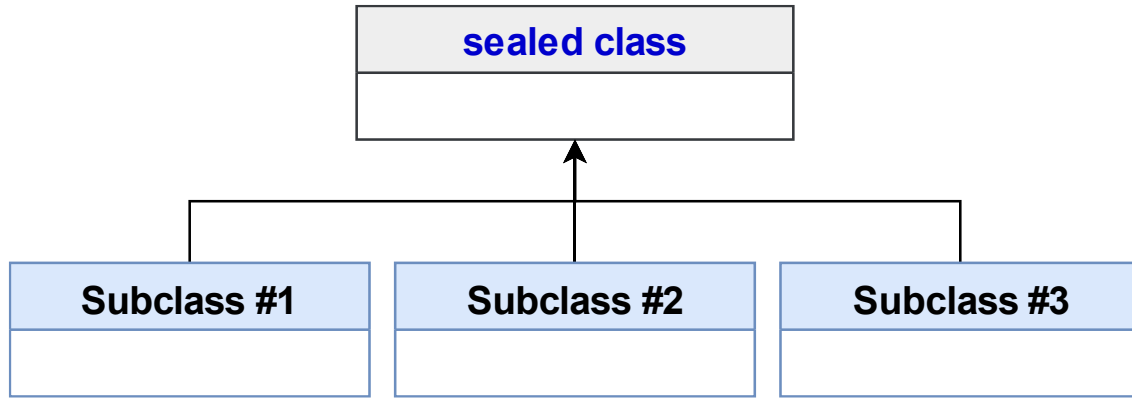


Java Runtime Environment 11.0.2+9-LTS

Computing the N=10,000th Fibonacci element.

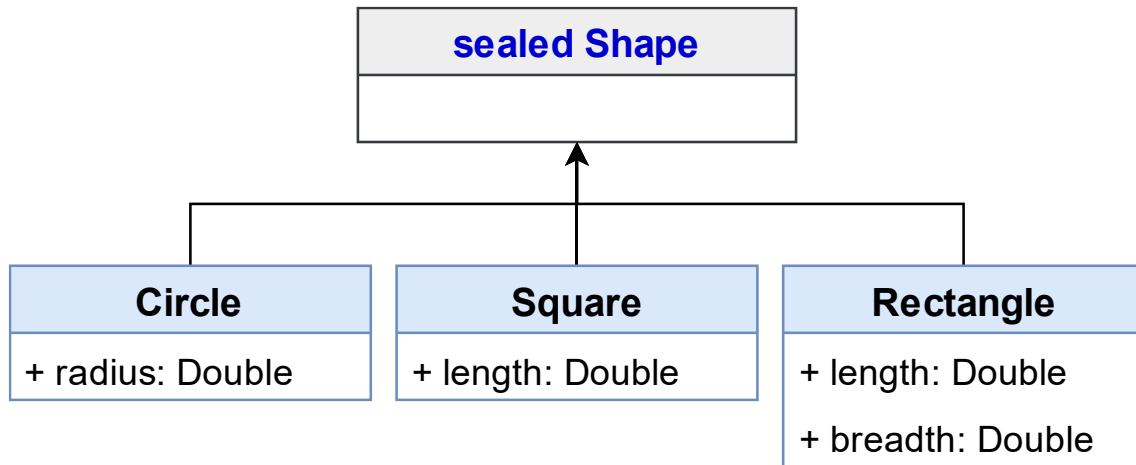
For the recursive function, performance is degrading with the size of N.

Sealed Classes



```
sealed class <className> {  
    // ... body ...  
}
```

Sealed classes represents restricted class hierarchies, when a value can have one of the types from a limited set, but cannot have any other type.



```
// return Shape area
return when (shape) {
    is Shape.Circle -> 3.14 * shape.radius * shape.radius
    is Shape.Square -> shape.length * shape.length
    is Shape.Rectangle -> shape.length * shape.breadth
    else -> throw RuntimeException("...") // or return DEFAULT_VALUE
}
```

```
double var10000; // Kotlin decompiled
if (shape instanceof Circle) {
    var10000 = 3.14D*((Circle)shape).getRadius()*((Circle)shape).getRadius();
} else if (shape instanceof Square) {
    var10000 = ((Square)shape).getLength()*((Square)shape).getLength();
} else {
    if (!(shape instanceof Rectangle)) {
        throw new NoWhenBranchMatchedException();
    }
    var10000 = ((Rectangle)shape).getLength()*((Rectangle)shape).getBreadth();
}
return var10000;
```